

Adobe Photoshop is Turing Complete

A. PUN, Not Adobe

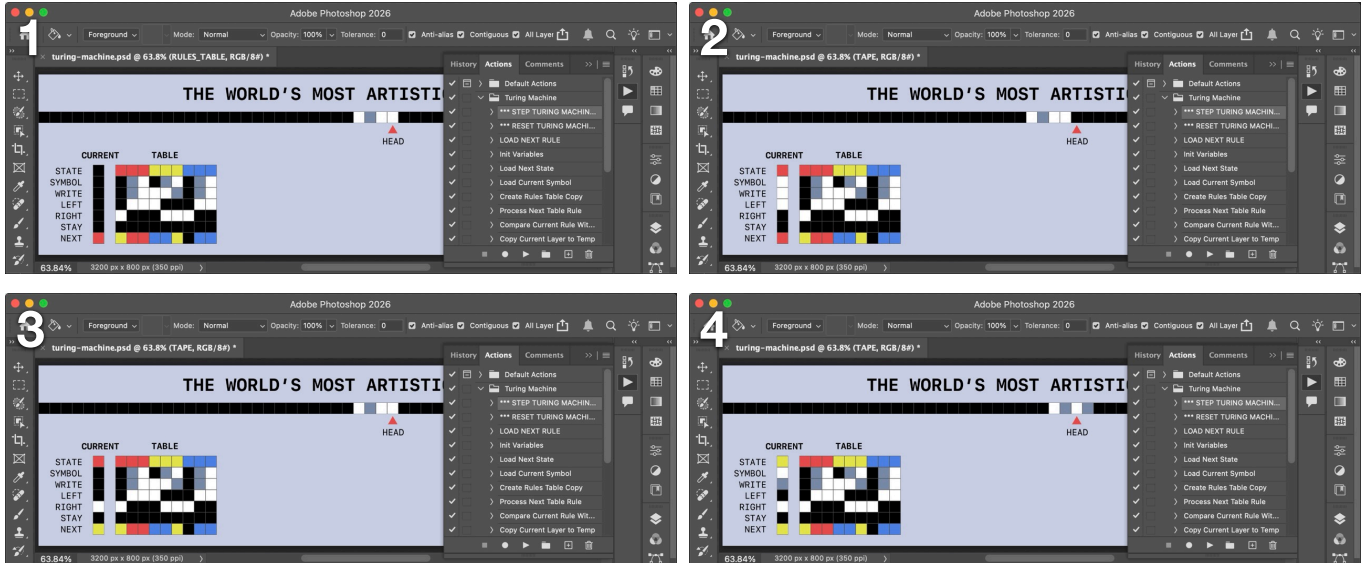


Fig. 1. A Turing machine implemented in Adobe Photoshop without the use of UXP scripting. The machine steps forward upon playing the “Step Turing Machine” Action. This figure depicts the first four execution steps of a program that adds 1 to its input.

Actions in Adobe Photoshop allow users to write simple scripts that play back a sequence of operations. As they lack conditional statements or detailed control flow, these scripts are suitable for parroting simple imperative tasks, but they cannot express more complex programs. Or can they?

In this paper, I demonstrate how to build a Turing machine using Photoshop Actions, showing the world that Actions are Turing complete and hence just as powerful as any normal programming language, like C++, Python, JavaScript, or Brainf*ck.

This paper is best enjoyed with animated figures. For a blog post containing said figures—and a link to download the Photoshop files and Actions so you can try them out yourself—see <https://avapun.com/blog/turing-machine-photoshop>. Or just imagine the pictures in this paper are moving. It’s almost the same.

1 Introduction

Adobe Photoshop is a popular raster graphics editor. It is typically not used to execute computer programs, as it is not a computer. But can it be a computer? The great and illustrious Alan Turing once proposed a Test for computers. An entity passes the Turing Test if one cannot distinguish it from a human. Humans can pass the Turing Test, while computers usually cannot. I can clearly distinguish Adobe Photoshop from a human, so it does not pass the Turing Test. Therefore, it can be a computer.

I show that Photoshop can indeed be a computer by implementing within it another invention of the great and illustrious Alan Turing: the Turing machine. This Turing machine does not use any of the JavaScript scripting features in Photoshop, because everyone knows JavaScript is already a computer. Instead, it uses Photoshop

Actions, which allow only for limited scripting with no loops or conditionals. To implement conditionals, it uses blending modes and the Threshold operation to compare pixel values between two layers. The result is a machine that can, in theory, run any computer program that has existed or will exist.

Let us begin.

2 Turing completeness

What is a computer? Four score and nine years ago, before computers even existed, the great and illustrious Alan Turing provided a definitive answer to this very question [Turing 1937]. According to the great and illustrious Alan Turing, a computer contains a *tape* that stretches forever in both directions. The tape is divided into infinitely many *cells*. Each cell is either blank or contains a *symbol*, typically the binary digit 0 or 1. A *tape head* points to the currently *active* symbol. Besides the tape, the computer stores a current *state*.

To program this computer, we provide it with a set of rules. Each rule corresponds to a (symbol, state) pair, and it specifies a new symbol, tape direction (left, right, or stay), and next state. Starting at a given state, the computer performs several steps. At each step, it chooses a rule to follow based on the current state and active symbol, then performs three actions according to the chosen rule:

- (1) Write the specified symbol in the active cell.
- (2) Move the tape in the specified direction.
- (3) Set the current state to the next state specified.

The computer continues stepping forward until it hits a given *halt* state. As it was invented by the great and illustrious Alan Turing, this kind of computer is known as a *Turing machine*.

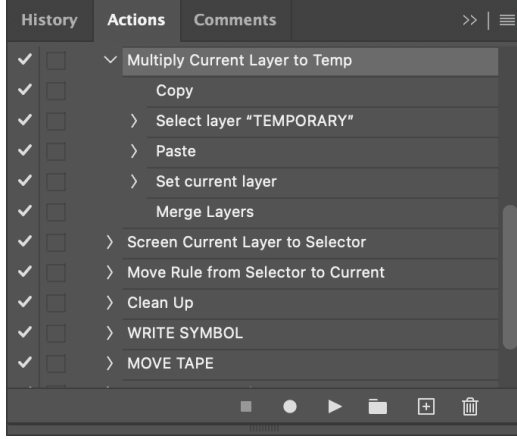


Fig. 2. The Actions panel in Photoshop displays a list of Actions (e.g., “Multiply Current Layer to Temp”). Each Action consists of several operations (e.g., “Copy”, “Select Layer”, “Paste”), which are executed when the Action is played. There are no loops or explicit if-statements.

In the 1970s, the home computer was discovered. It does not contain an infinitely long tape, hence proving the Turing machine to be a myth. However, the Turing machine remains an important and powerful model of computation. A system that can simulate a Turing machine is *Turing complete* and can, in theory, accomplish any task that can be accomplished by a computer, including compiling a $\text{\LaTeX}$ document, rendering a $\text{\LaTeX}$ document, and crashing during the writing of the Preliminaries section of said $\text{\LaTeX}$ document, forcing you to start all over again. Most programming languages are Turing complete, and as we shall see in this paper, so are Photoshop Actions.

3 Photoshop Actions

Adobe Photoshop [Adobe 2026] offers several automation tools to accelerate image editing workflows. One of these tools is *UXP scripting* [Adobe 2025c], which allows users to write JavaScript programs that execute operations performed within the software. However, JavaScript is a normal¹ programming language and already Turing complete, so we will not use scripting here. Instead, we will focus on a simpler automation tool: Photoshop *Actions* [Adobe 2025b].

Actions in Photoshop (fig. 2) allow users to record and play back a sequence of operations. That’s it. There are no loops or if-statements,² though Actions can be played from within other Actions. To simulate if-statements, we will perform operations on the pixel values of a Photoshop document. As a raster graphics editor, Photoshop is incredibly capable of performing such operations.

4 Pixel operations in Photoshop

A raster image can be represented by a two-dimensional grid of pixels. Each pixel contains a colour represented by a triplet of values (r, g, b) , which are floating-point numbers between 0 and 1 that

¹Opinions on the normality of JavaScript may vary.

²Today I learned that conditional actions do exist [Adobe 2023a]. However, they are limited to checking conditions from a short predefined list (e.g., the image orientation). In any case, we will not be using these to build our Turing machine.

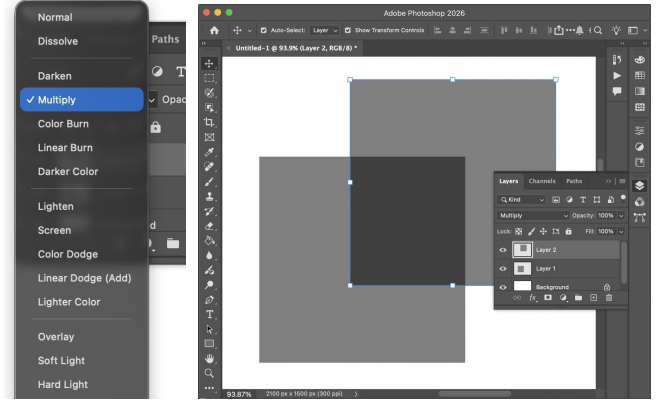


Fig. 3. By selecting the Multiply blending mode (left), the user can multiply two layers (right) with value 0.5 (50% grey), resulting in a value of 0.25 (25% grey).

denote the amount of red, green, and blue in the colour. Photoshop, like most good raster graphics editors, allows users to perform various binary and unary operations over these pixel values.

Binary operations. A Photoshop document consists of one or more *layers*, each containing a separate raster image. The pixel values on two layers can be combined by setting the top layer to a certain *blending mode* [Adobe 2025a]. There are many blending modes available, but the ones we will focus on are *Multiply* (fig. 3), which multiplies two pixel values; *Difference*, which takes the absolute difference between them; and *Screen*, which inverts the two values, multiplies them together, then inverts the result. In mathier words: $\text{MULTIPLY}(x, y) = xy$, $\text{DIFFERENCE}(x, y) = |x - y|$, and $\text{SCREEN}(x, y) = 1 - (1 - x)(1 - y)$. Note that these operations act channel-wise over the red, green, and blue channels; for example, if $x = (r_x, g_x, b_x)$ and $y = (r_y, g_y, b_y)$, then $xy = (r_x r_y, g_x g_y, b_x b_y)$. After setting the top layer’s blending mode, the result of the operation can be locked in by *merging* the two layers into a single layer containing the result.

Unary operations. Among the unary operations offered by Photoshop [Adobe 2023b] are *Invert*, which inverts the pixel values on a layer, and *Threshold*, which sets pixels to white if they are above a given threshold t , and black otherwise. In mathier words: $\text{INVERT}(x) = 1 - x$ and $\text{THRESHOLD}(x, t) = \mathbb{1}_{x \geq t}$.

5 Implementing a Turing machine using Photoshop Actions

I now describe how to implement a Turing machine with Photoshop Actions, starting from simple logic gates and building up to the final machine. Each step of the machine can be run by playing a single Action.

5.1 Creating logic gates

Computers are made of logic gates. I have a degree in computers, so you can trust me on this.



Fig. 4. One can construct AND and OR gates in Photoshop using the Multiply and Screen blending modes.

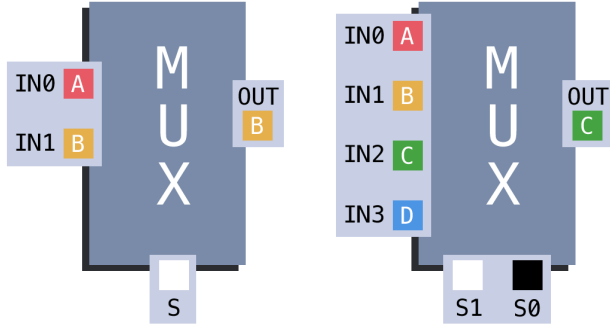


Fig. 5. A two-input multiplexor and a four-input multiplexor, implemented in Photoshop. Each can be run by playing an Action.

Since computers are made of logic gates, it makes sense to start by implementing logic gates with Photoshop Actions. We create Actions that perform the AND ($\wedge$), OR ($\vee$), and NOT ($\neg$) Boolean operations over pixel values on specified layers. (As a refresher, AND returns 1 if both of its inputs are 1, OR returns 1 if at least one of its inputs is 1, and NOT inverts its input: 1 becomes 0 and vice versa.) We use the Multiply and Screen³ blending modes to implement AND and OR, respectively (fig. 4), and we use Invert to implement NOT. In more detail, to simulate an AND (OR) gate, we record an Action that (1) copies its inputs to new layers and stacks them on top of each other, (2) sets the top layer's blending mode to Multiply (Screen), and (3) merges the two layers together. To simulate a NOT gate, we record an Action that simply copies its input to a new layer and applies the Invert operation. These logic gates, the building blocks of computation, can be chained together to make more complex components.

5.2 Building a multiplexor

One such component is a *multiplexor*, which outputs one of its two inputs depending on the value of a given selector. In our Photoshop file, we draw two input cells, IN0 and IN1; a binary output cell, OUT; and a binary selector cell, S (fig. 5). We want OUT to reflect IN0 if S is 0 (black), and IN1 if S is 1 (white). In other words, OUT should be 1 if and only if S is 0 and IN0 is 1 or S and IN1 are both 1. This can be described by the following Boolean expression:

$$\text{OUT} = (\neg S \wedge \text{IN0}) \vee (S \wedge \text{IN1}).$$

We evaluate this expression by chaining together the aforementioned logic gate actions, and we record the steps into a single multiplexor Action. In detail, the required operations are as follows:

³The Add blending mode can also be used to implement OR.

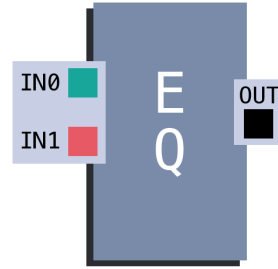


Fig. 6. An equality comparator implemented in Photoshop. Its output is white if its inputs are equal, and black otherwise.

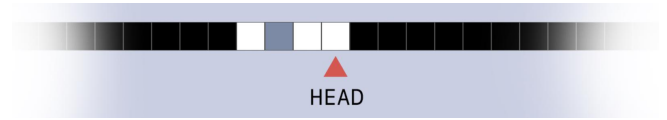


Fig. 7. The tape of our Photoshop Turing machine. This tape contains the binary string 1011, and the head points to the last character in the string.

- (1) Copy S to OUT, and Invert the copied S.
- (2) Copy IN0 to OUT, set the copied IN0 to Multiply, and merge it with the copied S to create $\neg S \wedge \text{IN0}$.
- (3) Copy S to OUT again.
- (4) Copy IN1 to OUT, set the copied IN1 to Multiply, and merge it with the copied S to create $S \wedge \text{IN1}$.
- (5) Set $S \wedge \text{IN1}$ to Screen, and merge it with $\neg S \wedge \text{IN0}$.

Note that this multiplexor functions correctly even if the input colours are not limited to black or white. Additionally, we can extend it to accept more than 2 inputs; implementation is left as an exercise to the reader. Multiplexors will be used in our Turing machine to select rules and move the tape according to a given rule.

5.3 Equality comparison

Another useful component is an *equality comparator*, which outputs 1 if its two input colours are equal and 0 otherwise. We create an equality comparator by drawing two input cells, IN0 and IN1, and a binary output cell, OUT (fig. 6). To run this comparator, we use the Difference blending mode to find the absolute difference between the inputs. We follow this with a Threshold to force the output to be 0 if the difference is 0, and 1 otherwise. Finally, we invert the result. In mathier words:

$$\text{EQ}(\text{IN0}, \text{IN1}) = \text{INVERT}(\text{THRESHOLD}(\text{DIFFERENCE}(\text{IN0}, \text{IN1}), \epsilon)).$$

We record these operations into a single comparator Action. Equality comparators will be used in our Turing machine to determine the correct rule to use for a given state and symbol.

5.4 The Turing machine

Now we can build our Turing machine in Photoshop! In our Photoshop file, we draw a tape (fig. 7) containing one colour in each cell. A grey cell indicates the symbol 0, a white cell indicates 1, and a black cell is blank. Alongside the tape, we draw a table (fig. 8) that

	CURRENT	TABLE
STATE	red	
SYMBOL	blue	
WRITE	blue	
LEFT	white	
RIGHT	black	
STAY	black	
NEXT	red	

Fig. 8. The rules table for our Turing machine, containing a program that adds 1 to the number on the tape. The currently active rule is stored on the far left. This particular rule corresponds to the state “red” and symbol 0.

contains the rules for the program we wish to run. Each column consists of seven coloured cells, which indicate the following:

- The STATE and SYMBOL associated with the rule.
- The new symbol to WRITE on the tape when following the rule.
- Three cells indicating whether the tape should go LEFT, go RIGHT, or STAY after following the rule, depending on which of these cells is coloured white.
- The NEXT state to move to, or a black cell if the program should halt.

Next to the rules table, we create a CURRENT column to store the currently active rule.

To run this Turing machine, we first specify the starting state by colouring the NEXT cell in the CURRENT column. Then, for each step of the machine, we do the following:

- (1) Use an equality comparator to determine which rule corresponds to the currently active STATE, which is in the NEXT cell, and SYMBOL, which is in the active tape cell.
- (2) Copy the selected rule into the CURRENT column, using the comparator’s results as the selection input to a multiplexor.
- (3) Copy the current WRITE symbol into the active tape cell.
- (4) Move the tape according to the current LEFT, RIGHT, and STAY values. This can be done by creating a left-shifted, a right-shifted, and a stationary copy of the tape, then choosing which copy to keep via a multiplexor that takes LEFT, RIGHT, and STAY as its selection input.

We record these moves into a “Step Turing Machine” Action. To run the Turing machine, the user simply plays the Action repeatedly until the halt state is reached (fig. 1). Any input program can be run in this way, including, in theory, Photoshop itself.

We have now proven that Adobe Photoshop can be a computer.

6 Limitations

As Photoshop Actions do not support loops, the Photoshop Turing Machine cannot step forward and halt automatically; the user must play the action for each step of the machine. Opinions are mixed on whether the inability to self-propel prevents it from being a “true” Turing machine. However, note that the PowerPoint Turing Machine [Wildenhain 2017] also lacks self-propulsion, and nobody’s complaining about that.

7 Related work

This paper is the latest in a long line of research in proving random things to be computers. Said things include C++ templates [Veldhuizen 2003], Java Generics [Grigore 2016], the x86 mov instruction [Dolan 2013], buggy JBIG2 image compression [Beer 2021], Conway’s Game of Life [Berlekamp et al. 1982], Wang tiles [Berger 1966], DOOM [Candelaria 2021], Dwarf Fortress [DF Wiki 2025], Baba Is You [Abel and Hendrickson 2024], Minecraft (okay, this one’s intentional), Magic: The Gathering [Churchill et al. 2019], Microsoft PowerPoint [Wildenhain 2017], and many more [Branwen 2022; Zwinkau 2023]. These are all great works, and you should go check them out, but not before perusing the remainder of this SIGBOVIK volume.

8 Discussion

I have presented a working Turing machine implemented in Adobe Photoshop. No further discussion is necessary.

Extensions. Though this paper uses Photoshop, the ideas within can be applied to any other raster graphics editor that supports actions, blending modes, and the Threshold operation. This means most graphics editors, except, sadly, Microsoft Paint.

References

- Zachary Abel and Della Hendrickson. 2024. Baba Is Universal. In *12th International Conference on Fun with Algorithms (FUN 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 291)*, Andrei Z. Broder and Tami Tamir (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 1:1–1:15. doi:10.4230/LIPIcs.FUN.2024.1
- Adobe. 2023a. Add conditional actions. <https://helpx.adobe.com/photoshop/using/conditional-actions-creative-cloud.html>.
- Adobe. 2023b. Apply special color effects to images. <https://helpx.adobe.com/photoshop/using/applying-special-color-effects-images.html>.
- Adobe. 2025a. Blending mode descriptions. <https://helpx.adobe.com/photoshop/desktop/repair-retouch/adjust-light-tone/blending-mode-descriptions.html>.
- Adobe. 2025b. Record an action. <https://helpx.adobe.com/photoshop/using/creating-actions.html>.
- Adobe. 2025c. Scripting. <https://helpx.adobe.com/photoshop/using/scripting.html>.
- Adobe. 2026. Official Adobe Photoshop. <https://www.adobe.com/products/photoshop>.
- Ian Beer. 2021. A deep dive into an NSO zero-click iMessage exploit: Remote Code Execution. <https://projectzero.google/2021/12/a-deep-dive-into-nso-zero-click.html>.
- Robert Berger. 1966. The undecidability of the domino problem. *Memoirs of the American Mathematical Society* 66 (1966), 72.
- Elwyn R. Berlekamp, John H. Conway, and Richard K. Guy. 1982. *Winning Ways for Your Mathematical Plays*. Vol. 2. Academic Press, New York.
- Gwern Branwen. 2022. Surprisingly Turing-Complete. <https://gwern.net/turing-complete>.
- Jared Candelaria. 2021. But can DOOM run it? <https://calabi-yau.space/blog/doom.html>.
- Alex Churchill, Stella Biderman, and Austin Herrick. 2019. Magic: The Gathering is Turing Complete. arXiv:1904.09828 [cs.AI] <https://arxiv.org/abs/1904.09828>
- DF Wiki. 2025. Computing. <https://dwarf fortress wiki.org/index.php/Computing>.
- Stephen Dolan. 2013. mov is Turing-complete. <https://drwho.virtadpt.net/files/mov.pdf>.
- Radu Grigore. 2016. Java Generics are Turing Complete. arXiv:1605.05274 [cs.PL] <https://arxiv.org/abs/1605.05274>
- Alan Turing. 1937. On Computable Numbers, with an Application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society* s2-42 (1937), 230–265. Issue 1.
- Todd L. Veldhuizen. 2003. C++ Templates are Turing-complete. <https://rtraba.com/wp-content/uploads/2015/05/cppturing.pdf>.
- Tom Wildenhain. 2017. On the Turing Completeness of MS PowerPoint. *SIGBOVIK* 11 (2017), 102–106.
- Andreas Zwinkau. 2023. Accidentally Turing-Complete. https://beza1e1.tuxen.de/articles/accidentally_turing_complete.html.

Received 4 March 2026